

DocuInsight - Technical Architecture Brief

Executive Summary

DocuInsight is an AI-powered Document Intelligence & Knowledge Management Platform designed to help Content and Knowledge Managers assess repository health, detect duplicative and conflicting information across documents, and perform pre-publish quality checks before adding new content to enterprise document repositories.

1. Project Objectives

Primary Goals

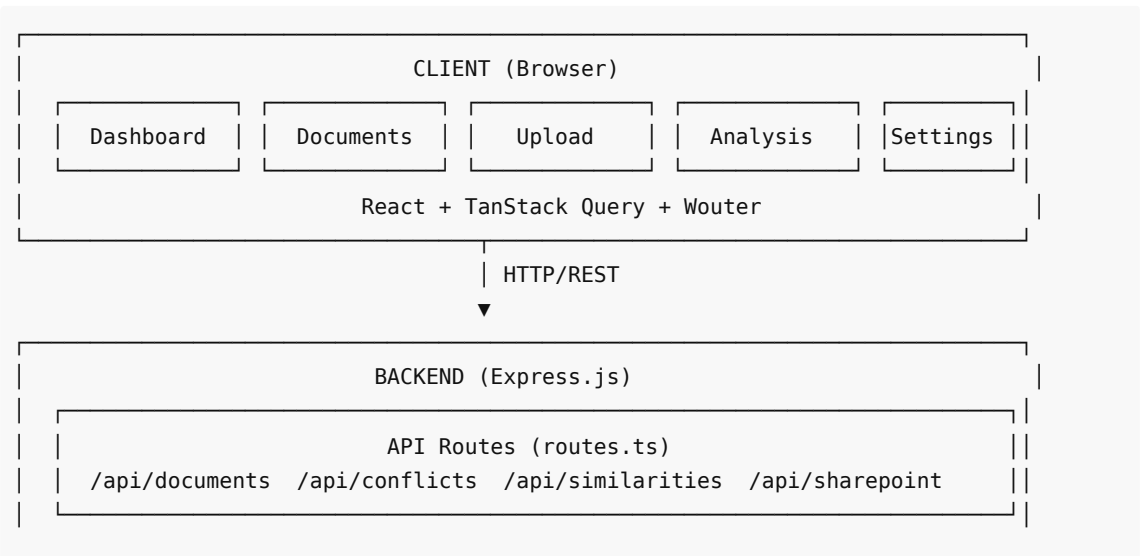
- Repository Health Assessment** - Provide visibility into the overall quality and organization of document repositories
- Duplicate Detection** - Identify semantically similar documents using AI-powered vector embeddings
- Conflict Detection** - Find contradictory information across documents (numerical, factual, date, entity conflicts)
- Pre-Publish Validation** - Check new documents against existing repository content before publication
- Location Recommendations** - Suggest optimal placement for new documents based on content analysis

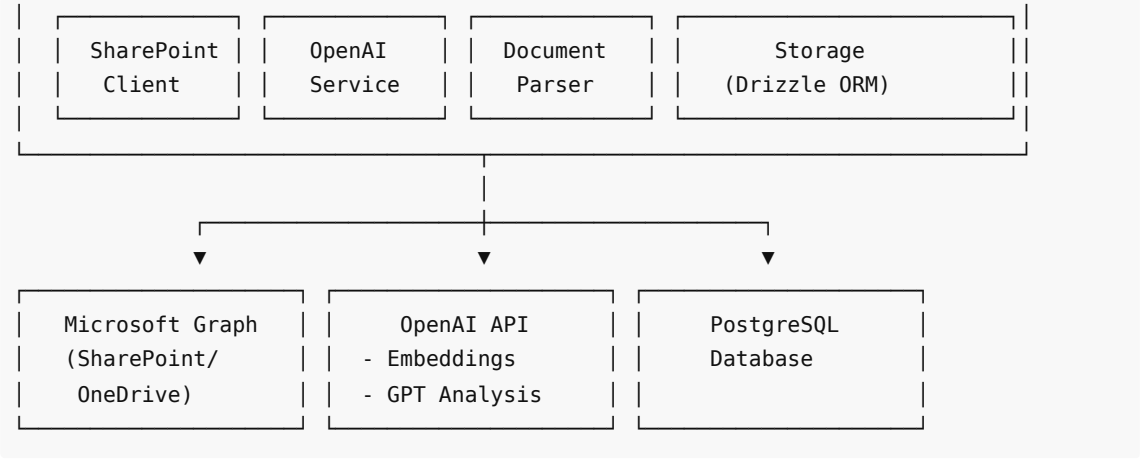
Target Users

- Content Managers
- Knowledge Managers
- Documentation Teams
- Enterprise Information Architects

2. System Architecture

2.1 High-Level Architecture





2.2 Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + TypeScript	UI Framework
	Vite	Build tool and dev server
	TanStack Query v5	Server state management
	Wouter	Client-side routing
	Tailwind CSS	Styling
	shadcn/ui	Component library
Backend	Node.js + Express	API server
	TypeScript	Type safety
	Drizzle ORM	Database access
Database	PostgreSQL (Neon)	Persistent storage
AI Services	OpenAI API	Embeddings & analysis
Integration	Microsoft Graph API	SharePoint/OneDrive access

3. Core Components

3.1 SharePoint Integration (server/sharepoint.ts)

Purpose: Connect to Microsoft SharePoint/OneDrive to sync documents from enterprise repositories.

Authentication Method: OAuth 2.0 Client Credentials Flow (Application Permissions)

Required Azure AD Permissions:

- Sites.Read.All (Application)
- Files.Read.All (Application)

Environment Variables:

```
MS_TENANT_ID    - Microsoft Entra tenant ID
MS_CLIENT_ID    - App registration client ID
MS_CLIENT_SECRET - App registration client secret
```

API Sequence for Document Sync:

```
1. GET /sites/{hostname}:/sites/{sitename} → site.id
2. GET /sites/{site-id}/drive              → drive.id
3. GET /drives/{drive-id}/root/children    → file list
4. GET /drives/{drive-id}/items/{item-id}/content → file binary
```

Key Functions:

- `getAccessTokenViaClientCredentials()` - Obtains OAuth token with caching
- `syncValueSite()` - Full sync operation for the configured site
- `getFileContent()` - Downloads file binary from SharePoint
- `isSharePointConnected()` - Connection health check

Token Caching: Access tokens are cached in memory with 60-second expiry buffer.

3.2 Document Parser (`server/document-parser.ts`)

Purpose: Extract text content from various document formats for AI analysis.

Supported Formats:

Format	MIME Type	Library
PDF	application/pdf	@cedrugs/pdf-parse
DOCX	application/vnd.openxmlformats-officedocument.wordprocessingml.document	mammoth
PPTX	application/vnd.openxmlformats-officedocument.presentationml.presentation	jszip (XML parsing)
XLSX	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	jszip (XML parsing)
TXT	text/plain	Native Buffer

Key Functions:

- `extractTextFromBuffer(buffer, mimeType)` - Main entry point for text extraction
- `extractPdfText()` - PDF parsing using pdf-parse
- `extractDocxText()` - Word document parsing using mammoth
- `extractPptxText()` - PowerPoint parsing via OOXML structure
- `extractXlsxText()` - Excel parsing with shared strings support
- `getMimeTypeFromFilename()` - MIME type detection from file extension

3.3 OpenAI Service (`server/openai.ts`)

Purpose: AI-powered document analysis using OpenAI's embedding and chat models.

Models Used:

- `text-embedding-3-small` - Vector embeddings (1536 dimensions)
- `gpt-4o-mini` - Similarity explanations (fast, cost-effective)
- `gpt-4o` - Conflict analysis and novelty assessment

Key Functions:

Embedding Generation

```
generateEmbedding(text: string): Promise<number[]>
```

Generates a 1536-dimensional vector embedding for semantic comparison.

Cosine Similarity

```
cosineSimilarity(a: number[], b: number[]): number
```

Calculates similarity score (0-1) between two embedding vectors.

Similarity Explanation (NEW)

```
explainSimilarity(doc1Content, doc2Content, similarityScore):  
Promise<SimilarityExplanation>
```

Uses GPT-4o-mini to generate human-readable explanations including:

- **Shared topics** - Common themes between documents
- **Shared facts** - Specific facts that appear in both
- **Key overlaps** - Most significant content overlaps
- **Differentiators** - What makes each document unique

Conflict Analysis

```
analyzeForConflicts(doc1Content, doc2Content, doc1Name, doc2Name):  
Promise<ConflictAnalysis[]>
```

Uses GPT to detect:

- **Numerical conflicts** - Different counts, amounts, percentages
- **Date conflicts** - Contradicting dates for events
- **Factual conflicts** - Inconsistent statements
- **Entity conflicts** - Different names/details for same entity

Novelty Assessment

```
analyzeNovelty(newDocContent, existingDocsContents): Promise<NoveltyAnalysis>
```

Returns novelty score (0-1) indicating how unique content is vs. existing repository.

Location Suggestion

```
suggestLocation(docContent, existingPaths): Promise<LocationSuggestion>
```

Recommends optimal folder placement based on content analysis.

3.4 Storage Layer (`server/storage.ts`)

Purpose: Database operations using Drizzle ORM with PostgreSQL + pgvector.

Interface: `IStorage` defines all CRUD operations.

Vector Storage & Indexing:

- Embeddings stored as `vector(1536)` type using pgvector extension
- **IVFFlat Index** for $O(\log n)$ approximate nearest neighbor searches
- Index parameters: `lists = 100` (optimized for ~200 documents)
- Similarity searches use `<=>` cosine distance operator

Key Operations:

- Document management (CRUD, search, filter by site)
- **Vector similarity search** - Native pgvector queries for finding similar documents
- **Document-centric aggregation** - Count duplicates/similar/conflicts per document
- Similarity tracking (store/retrieve pairwise scores)
- Conflict management (create, resolve, list)
- SharePoint site tracking
- Analysis job management
- Repository health calculation

Similarity Query Functions:

```
findSimilarDocuments(embedding, threshold, excludeDocumentId?):  
Promise<SimilarDocument[]>
```

Uses pgvector's IVFFlat index for efficient similarity search. The `excludeDocumentId` parameter prevents self-comparison in pre-publish checks.

3.5 API Routes (`server/routes.ts`)

Base Path: `/api`

Endpoint	Method	Purpose
<code>/api/health</code>	GET	Repository health metrics
<code>/api/documents</code>	GET	List all documents
<code>/api/documents/recent</code>	GET	Get 10 most recent documents
<code>/api/documents/:id</code>	GET	Get single document
<code>/api/documents/:id/analyze</code>	POST	Analyze single document
<code>/api/documents/pre-publish-check</code>	POST	Check file before publishing
<code>/api/conflicts</code>	GET	List all conflicts

/api/conflicts/recent	GET	Get recent conflicts
/api/conflicts/:id/resolve	PATCH	Mark conflict resolved
/api/similarities	GET	List document similarities
/api/analysis/documents	GET	Document-centric analysis (see below)
/api/analysis/run	POST	Run batch analysis job
/api/sharepoint/sites	GET	List synced sites
/api/sharepoint/sync	POST	Sync from SharePoint
/api/settings/status	GET	Connection status check

Document-Centric Analysis Endpoint (/api/analysis/documents):

Query parameters:

- `category` - Filter by: `duplicates` , `similar` , `novel` , or `conflicts`
- `limit` - Number of documents to return (default: 100)
- `offset` - Pagination offset

Category definitions:

Category	Criteria
duplicates	Documents with ≥ 1 similarity $\geq 80\%$
similar	Documents with ≥ 1 similarity 50-79%
novel	Documents with noveltyScore < 0.3
conflicts	Documents with ≥ 1 unresolved conflict

Response includes aggregated counts per document:

```
{
  data: Array<Document & {
    duplicateCount: number; // Count of docs with  $\geq 80\%$  similarity
    similarCount: number;   // Count of docs with 50-79% similarity
    conflictCount: number;  // Count of unresolved conflicts
  }>;
  total: number;
  limit: number;
  offset: number;
}
```

4. Data Model

4.1 Database Schema (`shared/schema.ts`)

```

|-----<| document_similarities |>-----| documents |-----|
|-----|-----| id (PK) | | id (PK) | | name | | document_id_1
(FK) | | path | | document_id_2 (FK) | | site_id | | similarity_score | | drive_id | | similar_sections |
| item_id (unique) | | analyzed_at | | mime_type |-----| size | | content
(text) |-----| embedding (json) |-----<| conflicts | | novelty_score |
|-----| health_score | | id (PK) | | analyzed_at | | document_id_1 (FK) | |
created_at | | document_id_2 (FK) | | modified_at | | conflict_type |-----|
severity | | description |-----| context_1/2 | | sharepoint_sites | | value_1/2 |
|-----| resolved | | id (PK) | | detected_at | | site_id (unique) |
|-----| site_name | | site_url |-----| last_sync_at | |
analysis_jobs | | created_at |-----|-----| id (PK) | | status |
| document_ids (json) | | progress | | total_documents | | error | | started_at | | completed_at |
|-----|

```

4.2 Key Types

```

```typescript
// Document with embedding
interface Document {
 id: string;
 name: string;
 path: string;
 embedding: number[] | null; // 1536-dim vector
 noveltyScore: number | null; // 0-1 scale
 content: string | null;
 // ... metadata fields
}

// Conflict between documents
interface Conflict {
 documentId1: string;
 documentId2: string;
 conflictType: 'factual' | 'numerical' | 'date' | 'entity';
 severity: 'critical' | 'warning' | 'info';
 description: string;
 resolved: boolean;
}

// Pre-publish check result
interface PrePublishCheckResult {
 novelty: number;
 conflicts: Conflict[];
 similarDocuments: Array<{ document: Document; similarity: number }>;
 suggestedLocation: string;
 recommendation: 'publish' | 'review' | 'reject';
 issues: string[];
}

```

## 5. Frontend Architecture

### 5.1 Page Structure

Route	Page	Purpose
/	Dashboard	Health metrics, recent activity
/documents	Documents	Document list, sync controls
/upload	Upload	Pre-publish document checker
/analysis	Analysis	Conflicts and duplicates view
/settings	Settings	Connection management

### 5.2 Design System

Based on **Microsoft Fluent Design System** (per `design_guidelines.md`):

- Enterprise-focused color palette
- Clean typography with Inter font
- Consistent spacing (4px grid)
- Professional, accessible UI

### 5.3 State Management

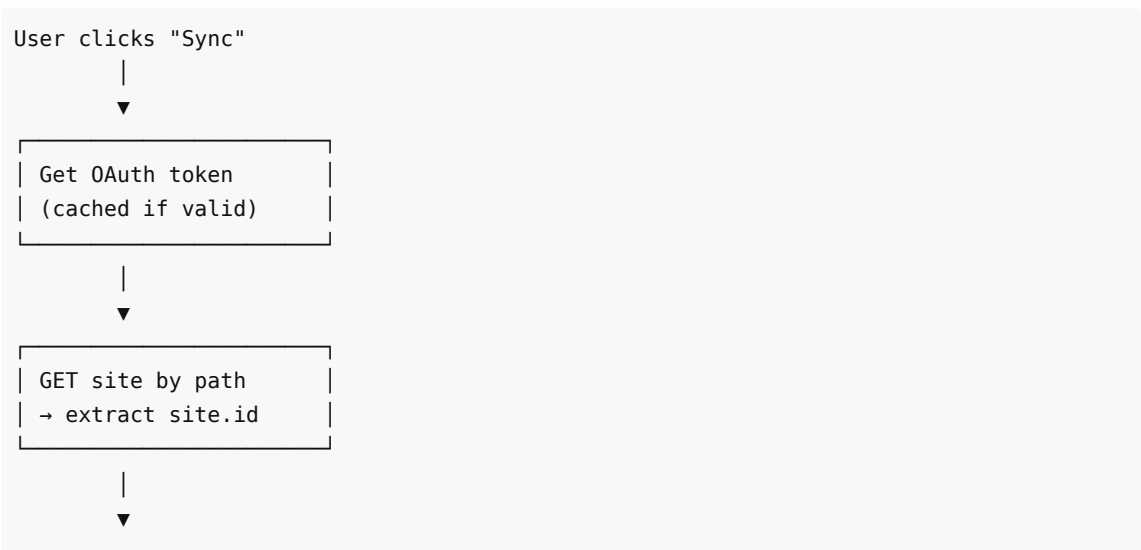
**TanStack Query** handles all server state:

- Automatic caching and invalidation
- Background refetching
- Loading/error states
- Mutation handling with cache updates

---

## 6. Core Workflows

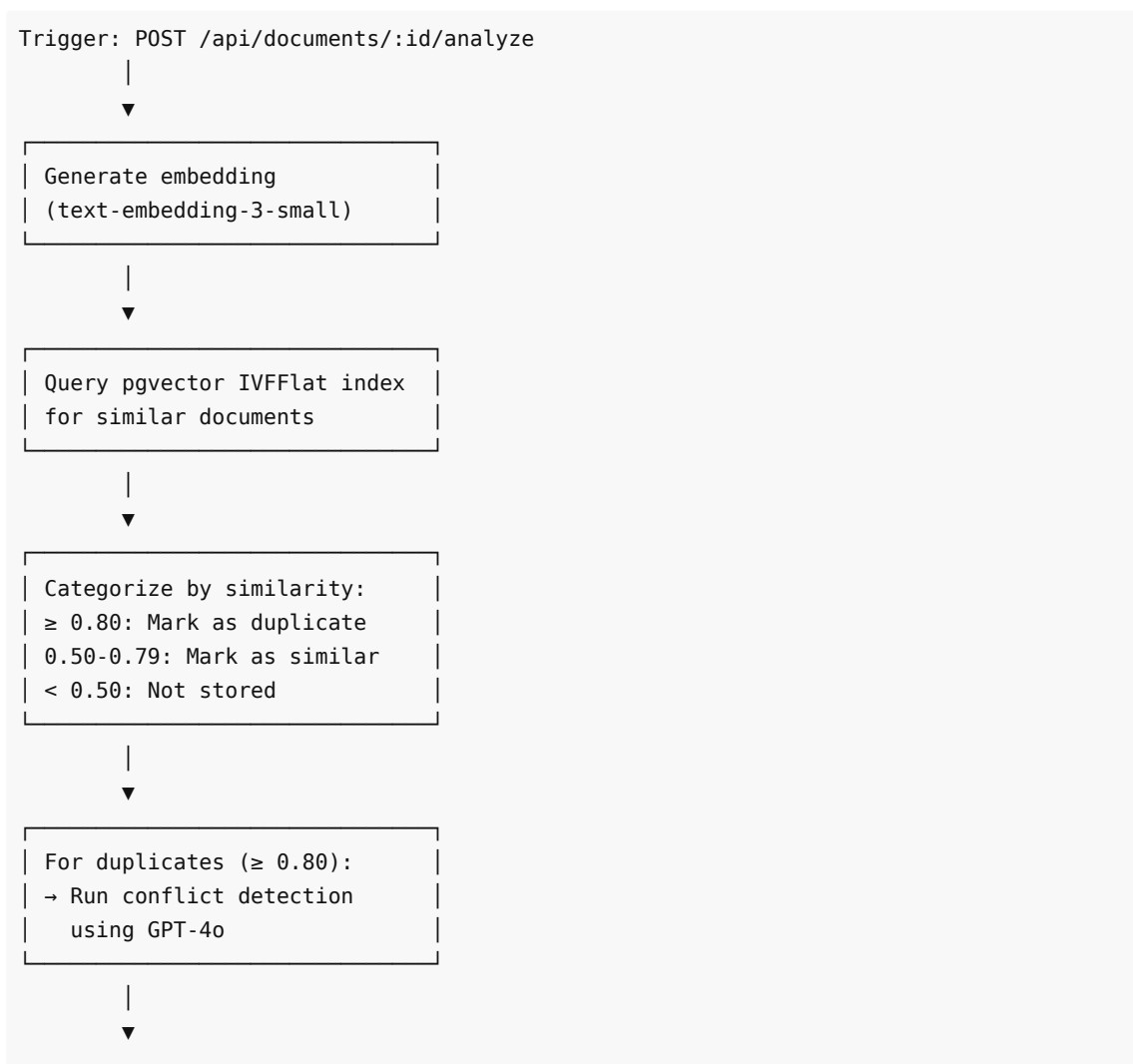
### 6.1 SharePoint Sync Flow

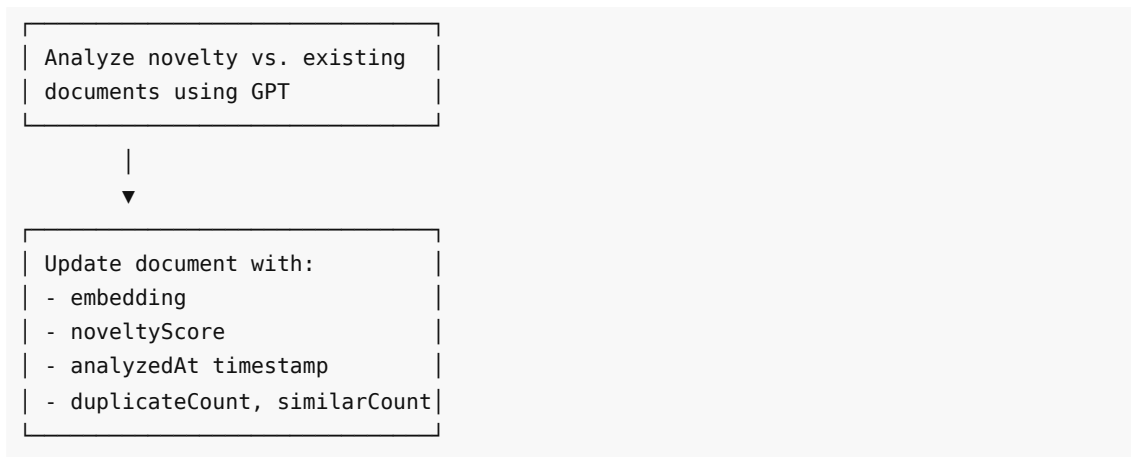






## 6.2 Document Analysis Flow





**Document-Centric Aggregation:** After analysis, each document has aggregated counts:

- `duplicateCount` - Number of documents with  $\geq 80\%$  similarity
- `similarCount` - Number of documents with 50-79% similarity
- `conflictCount` - Number of unresolved conflicts

These counts power the Analysis page tabs.

### 6.3 Pre-Publish Check Flow





**Pre-Publish UI Features:**

- Expandable cards showing similar documents
- AI-generated explanations with shared topics, facts, and differentiators
- Color-coded similarity badges (red for duplicates, yellow for similar)
- Conflict warnings with severity indicators

## 7. Similarity & Conflict Detection

### 7.1 Similarity Scoring

**Threshold Levels** (Updated):

Score	Category	Interpretation	Action
≥ 0.80	<b>Duplicate</b>	Near-duplicate content	Shown in Duplicates tab
0.50-0.79	<b>Similar</b>	Related content	Shown in Similar tab
0.30-0.49	Low similarity	Some overlap	Stored but not highlighted
< 0.30	Distinct	Minimal relationship	Not stored

**Analysis Page Categories:**

- **Duplicates Tab:** Documents having at least one similarity ≥ 80%
- **Similar Content Tab:** Documents with similarities in 50-79% range (excluding duplicates)
- **Novel Documents Tab:** Documents with noveltyScore < 0.3 (highly unique)
- **Conflicts Tab:** Documents with unresolved conflicts

**pgvector Performance:**

- IVFFlat index provides  $O(\log n)$  query time vs  $O(n)$  brute force
- Cosine distance operator `<=>` used for similarity calculation
- Similarity =  $1 - \text{distance}$  (converted from distance to score)

**7.2 Conflict Types**

Type	Description	Example
numerical	Different numbers	"Revenue: \$1M" vs "\$2M"
date	Conflicting dates	"Founded 2019" vs "2020"
factual	Contradicting facts	"CEO is John" vs "CEO is Jane"
entity	Different entity details	Different addresses

**7.3 Severity Levels**

Level	Impact	Action
critical	Major contradiction	Block publishing
warning	Potential issue	Review required
info	Minor discrepancy	FYI only

---

**8. Current State**

**8.1 Completed Features**

- SharePoint integration with ValueSite sync (199 documents synced)
- Document parsing for PDF, DOCX, PPTX, XLSX, TXT
- OpenAI embeddings generation (text-embedding-3-small)
- **pgvector IVFFlat index** for  $O(\log n)$  similarity searches
- **Document-centric Analysis page** with category tabs (Duplicates, Similar, Novel, Conflicts)
- **AI-powered similarity explanations** using GPT-4o-mini
- Pre-publish document checking with expandable detail cards
- Conflict detection with GPT-4o analysis
- Novelty scoring with excludeDocumentId for accurate self-exclusion
- Repository health dashboard
- Dark/light theme support
- Full CRUD for documents, conflicts, similarities

**8.2 Key Metrics (as of 2024-12-31)**

- Total documents: 199
- High duplicates ( $\geq 80\%$ ): 2,408 pairs
- Similar content (50-79%): 12,599 pairs
- Novel documents (noveltyScore < 0.3): 15
- Unresolved conflicts: 0

**8.3 Environment Configuration**

**Required Secrets:**

MS_TENANT_ID	- Microsoft Entra tenant ID
MS_CLIENT_ID	- Azure AD app client ID
MS_CLIENT_SECRET	- Azure AD app client secret
OPENAI_API_KEY	- OpenAI API key
SESSION_SECRET	- Express session secret
DATABASE_URL	- PostgreSQL connection string

**9. File Structure**

```
/
├── client/
│ └── src/
│ ├── components/ # Reusable UI components
│ │ └── ui/ # shadcn components
│ ├── hooks/ # Custom React hooks
│ ├── lib/ # Utilities (queryClient, utils)
│ ├── pages/ # Page components
│ │ ├── dashboard.tsx
│ │ ├── documents.tsx
│ │ ├── upload.tsx
│ │ ├── analysis.tsx
│ │ └── settings.tsx
│ ├── App.tsx # Main app with routing
│ └── index.css # Global styles + CSS variables
├── server/
│ ├── db.ts # Database connection
│ ├── document-parser.ts # Text extraction
│ ├── index.ts # Server entry point
│ ├── openai.ts # OpenAI integration
│ ├── routes.ts # API endpoints
│ ├── sharepoint.ts # Microsoft Graph client
│ ├── storage.ts # Database operations
│ └── vite.ts # Vite middleware
├── shared/
│ └── schema.ts # Database schema + types
├── design_guidelines.md # UI design system
├── replit.md # Project documentation
└── package.json # Dependencies
```

**10. Dependencies**

**Production Dependencies**

Package	Version	Purpose
@microsoft/microsoft-graph-client	^3.0.x	SharePoint API
@cedrugs/pdf-parse	^1.x	PDF text extraction

mammoth	^1.x	DOCX text extraction
jszip	^3.x	OOXML parsing (PPTX/XLSX)
openai	^4.x	OpenAI API client
drizzle-orm	^0.x	Database ORM
pg	^8.x	PostgreSQL driver
express	^4.x	HTTP server
multer	^1.x	File upload handling

Frontend Dependencies

Package	Version	Purpose
react	^18.x	UI framework
@tanstack/react-query	^5.x	Server state
wouter	^3.x	Routing
tailwindcss	^3.x	Styling
lucide-react	^0.x	Icons

## 11. Security Considerations

- 1. **Authentication:** Client credentials flow with Azure AD
- 2. **Token Management:** Cached with expiry, never exposed to frontend
- 3. **File Handling:** Memory-based upload with multer
- 4. **Input Validation:** Zod schemas for API validation
- 5. **SQL Injection:** Prevented by Drizzle ORM parameterization
- 6. **Secrets:** Stored in environment variables, never committed

## 12. Performance Notes

- 1. **Token Caching:** Reduces auth overhead
- 2. **Batch Embeddings:** Support for bulk embedding generation
- 3. **Background Jobs:** Analysis runs asynchronously
- 4. **Pagination:** Recent documents limited to 10
- 5. **Content Truncation:** Text limited to 8000 chars for embeddings